

TECHNICAL REPORT | VERSION 1.1 | SEPTEMBER 2026

Smart Helmet AIoT

A Reproducible Edge-to-Cloud Safety Monitoring System

TuoKe Ke | ketk0917@163.com | github.com/Kettkk

Evidence baseline: v1.0.0 release, commit 716644f. Version 1.1 adds reporting context only.

Abstract

This report presents a reproducible research prototype for studying an edge-to-cloud monitoring pipeline derived from a physical smart-helmet project. The system combines MQTT telemetry, Spring Boot and MySQL persistence, a Vue dashboard, fixed-video person detection, and repeatable network experiments. Its scope is deliberately limited to latency, throughput, sampled-frame detection continuity, and delivery reliability. On a 20-second reference video, frame stride 5 reached 34.56 pipeline FPS with 99.17% sampled-frame continuity. Across nine broker-outage trials, every client recovered after broker availability.

Research question

How do frame-sampling frequency and network conditions affect the latency, throughput, reliability, and detection continuity of a resource-constrained edge-to-cloud monitoring pipeline?

Smart Helmet AIoT — implemented public architecture

A live telemetry loop and a controlled offline vision experiment, with archived hardware shown only as provenance

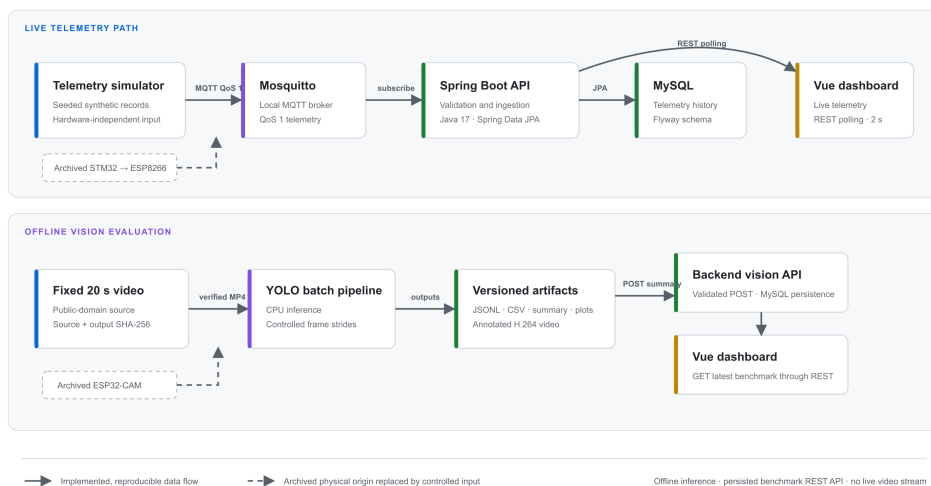


Figure 1. Public reproducibility architecture. Telemetry and fixed-video results converge in the backend and dashboard.

System design and implementation

Telemetry loop

A simulator or the archived ESP8266 bridge publishes telemetry through MQTT QoS 1. Mosquitto forwards messages to Spring Boot; Flyway-managed MySQL tables persist them; REST endpoints expose the latest state to the Vue dashboard.

Vision loop

A downloader restores a public-domain 20-second video and checks its SHA-256 digest. YOLO11n produces JSONL, CSV, annotated video, and benchmark summaries. A publication script posts the summary to the backend, which stores it and serves the latest result to the dashboard.

Physical prototype provenance

The helmet used an ESP32-CAM for imaging and an ESP8266 as a UART-to-network bridge. Sensor values originated from an STM32 subsystem developed by another contributor. Hardware images and authored firmware are retained as provenance; public tests use simulators.

Reproducibility boundary

The public workflow validates software behavior without requiring the helmet. It does not represent medical validation, a certified safety product, radio-layer packet loss, or detector accuracy. This boundary keeps every reported claim tied to versioned code and observable measurements.

Implementation map

Layer	Implementation
Messaging	Eclipse Mosquitto, MQTT QoS 1
Backend	Java, Spring Boot, Flyway
Storage	MySQL
Dashboard	Vue, Vite
Vision	Python, OpenCV, YOLO11n
Packaging	Docker Compose
Verification	JUnit, pytest, smoke tests

Independent contributions

- Public edge-to-cloud architecture and Docker workflow
- ESP8266 parsing and cloud integration, plus ESP32-CAM integration
- Telemetry ingestion, persistence, REST APIs, and dashboard
- Fixed-input vision benchmark and backend publication
- Frame-sampling and network-reliability experiments
- Scope, provenance, limitations, and repeatability documentation

Frame sampling: latency, throughput, continuity

The fixed input contains 600 frames at 1280 x 720 and 30 FPS. YOLO11n runs on CPU at image size 640 with confidence threshold 0.25. Continuity is the percentage of sampled frames containing at least one person detection; it is not an accuracy metric.

Frame-sampling trade-off on the fixed hiking video

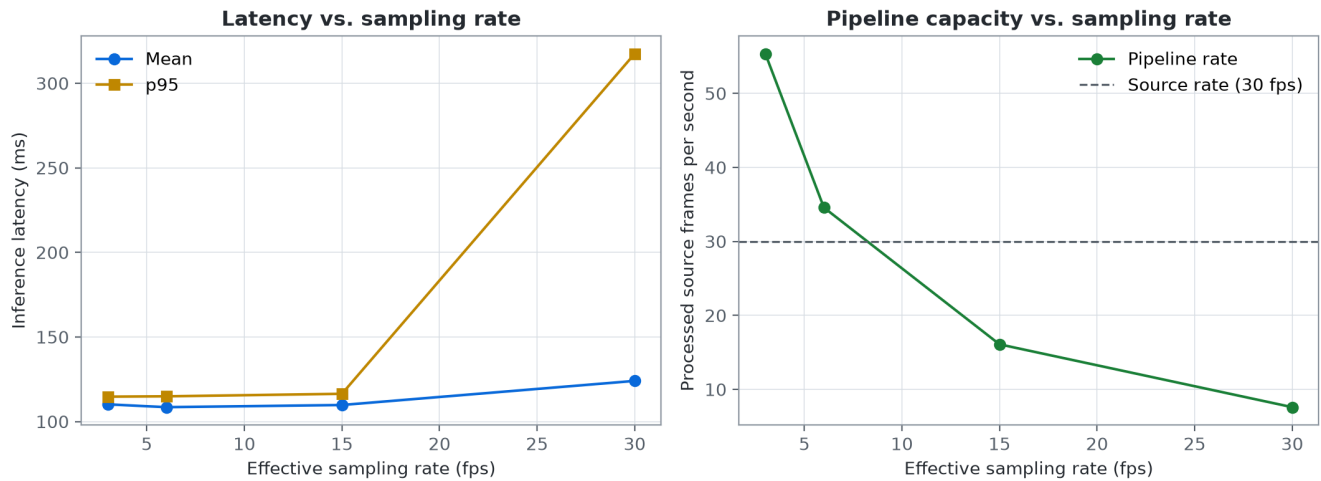


Figure 2. Measured frame-sampling trade-offs on the fixed reference clip.

Stride	Sampling	Mean latency	p95 latency	Pipeline	Continuity
1	30 FPS	124.00 ms	317.43 ms	7.59 FPS	99.50%
2	15 FPS	109.75 ms	116.37 ms	16.08 FPS	99.33%
5	6 FPS	108.47 ms	114.88 ms	34.56 FPS	99.17%
10	3 FPS	110.14 ms	114.68 ms	55.35 FPS	98.33%

Interpretation

Stride 1 did not keep pace with the 30 FPS source on the evaluated CPU path. Stride 5 processed the sampled workload at 34.56 pipeline FPS while retaining 99.17% continuity. Stride 10 increased throughput further, but continuity fell to 98.33%. Frame sampling is therefore a useful load-control mechanism for this clip, but continuity must be measured rather than assumed.

Network reliability and conclusions

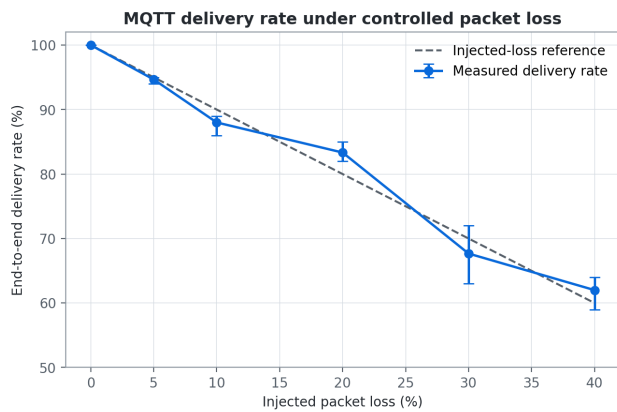


Figure 3. Attempted-message delivery under seeded loss.

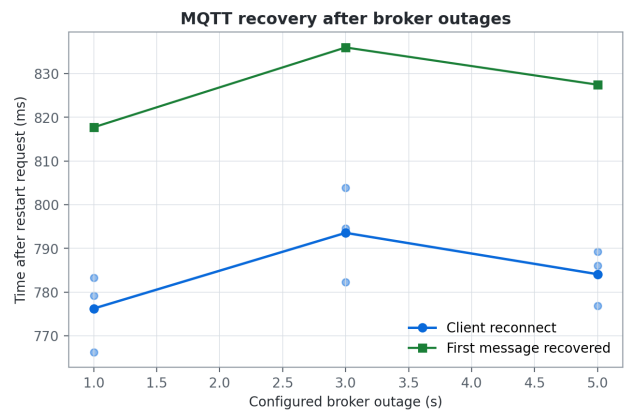


Figure 4. Recovery after broker availability.

Experiment	Conditions	Observed result
Added delay	0-200 ms, 3 x 20 messages	2.74-207.91 ms mean latency
Seeded loss	0%-40%, 3 x 100 attempts	100.00%-62.00% delivery
Broker outage	1, 3, 5 s, 3 repeats	9/9 recovered; 0.78 s subscription mean

Limitations

- Offline fixed-video inference, not a live ESP32-CAM stream
- No annotated ground truth; continuity is not precision or recall
- Single-host Docker trials with application-level loss
- Simulators replace physical devices in automated tests
- No long-duration field deployment or safety certification

Conclusion

This release demonstrates a complete, inspectable path from telemetry and fixed-video inputs to persistent backend records and a user-facing dashboard. Its strongest evidence is not code volume, but reproducibility: fixed inputs, raw results, regenerable plots, tests, explicit provenance, and bounded claims.

Next evaluation

Longer repeated runs, confidence intervals, Linux tc netem validation, controlled physical-device trials, and adaptive sampling evaluated only against latency, throughput, continuity, and reliability.

Threats to validity and references

Internal validity

The benchmark runner fixes the video, model, image size, threshold, frame strides, random seed, and message counts. This supports repeatability, but a fixed clip and one CPU path cannot remove implementation- or machine-specific effects. Versioned JSONL, CSV, plots, and configurations make each reported value inspectable.

Construct validity

Sampled-frame continuity means that a sampled frame contains at least one person detection. It is not precision, recall, mAP, or a safety outcome because the clip has no human-annotated ground truth. Seeded application-level loss measures attempted-message delivery, not radio-layer packet loss. The report uses these restricted names throughout.

External and conclusion validity

One 20-second hiking clip, one local Docker host, and simulator-driven telemetry do not represent all lighting, motion, hardware, wireless conditions, or deployment durations. Network settings have three repeats and the vision study is a deterministic fixed-input comparison, not a confidence-interval study. The results support directional trade-offs only in this prototype setting.

Research artifact

Reported measurements are pinned to Smart Helmet AIoT release v1.0.0, commit 716644f (<https://github.com/Kettkk/smart-helmet-aiot/releases/tag/v1.0.0>). This v1.1 report updates author, artifact, validity, and reference context without changing the experimental baseline.

References

- 1. OASIS. MQTT Version 5.0. OASIS Standard, 2019. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- 2. Jocher, G., and Qiu, J. Ultralytics YOLO11. Software documentation, 2024. <https://docs.ultralytics.com/models/yolo11/>
- 3. Kettkk. Smart Helmet AIoT v1.0.0. GitHub release and versioned experiment artifact, 2026. <https://github.com/Kettkk/smart-helmet-aiot/releases/tag/v1.0.0>